

Automated Freshman Course Scheduling Model for RIT

Priyaa Gopal Shankar

Department of Computer Science

Golisano College of Computing and Information Sciences

Rochester Institute of Technology

Rochester, NY 14623

pg5918@rit.edu

Varsha Dani, Advisor

Department of Computer Science

Golisano College of Computing and Information Sciences

Rochester Institute of Technology

Rochester, NY 14623

vxdvcs@rit.edu

Abstract—Scheduling courses at a university is tough because you’re dealing with thousands of students every semester. This project built an automated system for RIT that handles freshman course scheduling. The system checks data quality first, then assigns students to course sections without creating time conflicts, going over capacity, or missing program requirements. It uses a two-step process: validation catches problems early (capacity shortages, impossible schedules), then a CP-SAT solver finds assignments. There’s also an LLM interface powered by Claude Sonnet 4 so administrators can write rules in normal English rather than code. I tested it on synthetic data based on RIT’s 44 programs and 4,155 freshmen. The system got 100% of students scheduled with no conflicts and reasonably balanced sections.

Index Terms—Constraint Programming; Course Scheduling; Optimization; Large Language Models; University Timetabling

I. INTRODUCTION

Every semester, universities deal with a huge scheduling problem: figuring out how to get incoming freshmen into their required courses. At RIT, with 44 different academic programs and over 4,000 new students, this means assigning each student to specific class sections without creating schedule conflicts, going over classroom capacities, or forgetting about different math placement levels. The old way—doing all this by hand—eats up a ton of time and doesn’t even guarantee you’re using resources well.

The freshman course scheduling problem is a version of the well-studied university timetabling problem. The problem is computationally hard [3]—there are too many possible assignment combinations and too many constraints to satisfy simultaneously. What makes freshman scheduling different: students in the same program all need the same courses, students have different math placement levels that need individual handling, and the goal is assigning students to existing sections rather than creating a master timetable from scratch. At scale, this creates hundreds of thousands of decision variables, making manual scheduling impractical.

To understand the scale of this problem, consider a Computer Science freshman who must enroll in CSCI 141 (Computer Science I), a math course based on their placement exam, and several other program requirements. There are 26 sections of CSCI 141 across various time slots and 83 sections of calculus-level mathematics. This creates thousands

of possible schedule combinations. With 250 Computer Science freshmen—each needing different math courses based on placement—the number of valid assignments explodes. Across 44 programs, this results in hundreds of thousands of decision variables and constraints.

Inefficient scheduling creates real headaches. The manual process takes weeks of trial and error to find valid assignments. Staff can usually get conflict-free schedules eventually, but they’re not optimal: some sections end up packed while others sit half-empty, which wastes classroom space and teaching resources. Plus, it doesn’t scale. Each new program or course makes the problem exponentially harder, and trying to add new scheduling rules means basically starting over.

This project addresses the freshman scheduling problem through an automated approach that combines constraint programming with data validation. The system has three main goals:

- 1) **Feasibility:** Every student gets a complete, conflict-free schedule that meets all their program requirements.
- 2) **Capacity Compliance:** No section can be overfilled beyond its classroom capacity.
- 3) **Quality Optimization:** Among all valid schedules, prefer ones that balance section loads rather than having some sections packed and others nearly empty.

The technical approach uses Google OR-Tools’ CP-SAT solver, which can handle problems with hundreds of thousands of variables. The scheduling problem is modeled as a constraint satisfaction problem: decision variables represent student-section assignments, and constraints encode things like time conflicts, capacity limits, and program requirements. The solver uses techniques like clause learning, restarts, and linear relaxation to find good solutions efficiently.

The system makes several contributions to automated course scheduling. The data validation module goes beyond simple capacity checks. It implements n -way-conflict detection—a validation algorithm that checks whether all n required courses for a program can be scheduled simultaneously without time conflicts, where n represents the number of courses required by that program—catching problems that basic validation would miss. It also performs effective capacity analysis, accounting for time overlap constraints to identify scheduling

bottlenecks where limited non-conflicting sections restrict assignments. Additionally, the LLM interface makes the system accessible to non-technical users. Academic scheduling often involves requirements like “Limit Engineering students to no more than 60% of any PHYS 211 section” that traditionally need programming expertise to implement. The LLM-based interface (powered by Anthropic’s Claude Sonnet 4) interprets natural language preferences and generates the appropriate constraint code.

The system works in two phases with an LLM layer on top. The validation phase runs first, catching problems in the input data before the expensive optimization starts. Then the constraint solver builds and solves the model to generate student assignments. The LLM interface lets administrators add custom constraints in plain English instead of writing code.

Evaluation uses synthetic data designed to test the system: 44 academic programs modeled on RIT’s structure, 634 course sections across 148 courses, and 4,155 simulated freshmen with different math placement levels. The program requirements match RIT’s actual curriculum, but the enrollment numbers, section counts, and meeting times are synthetic to create a realistic test case. Performance is measured by feasibility rate, time conflicts, section load balance, and solver runtime.

The remainder of this report is organized as follows. Section II reviews related work in university timetabling, constraint programming applications, and LLM integration in optimization systems. Section III details the system architecture, data models, and constraint formulations. Section IV describes the implementation, including the validation pipeline, solver configuration, LLM interface, and Power BI dashboard for result visualization. Section V presents experimental results including validation performance, scheduling system performance, and dashboard visualizations. Section VI discusses limitations and directions for future work. Section VII concludes with a summary of contributions.

II. BACKGROUND AND RELATED WORK

A. University Timetabling Problem

The university timetabling problem is about assigning courses, instructors, students, and rooms to time slots while satisfying institutional constraints. Even et al. [3] proved this problem is NP-hard, meaning no polynomial-time algorithm can guarantee optimal solutions for arbitrary instances. This has motivated decades of research into methods for producing good schedules within reasonable time limits.

Schaerf [6] surveys automated timetabling and breaks the problem into three main variants: school timetabling, course timetabling, and examination timetabling. A more recent survey by Ceschia et al. [8] identifies six standard formulations and reports current best results.

A less-studied variant is the *student sectioning problem*. Here, the master timetable of course sections already exists—the problem is just assigning individual students to specific sections of their required courses. Müller and Murray [9]

worked on this at Purdue University with over 9,000 sections, showing that student sectioning at this scale needs careful algorithm design to balance capacity constraints, avoid time conflicts, and distribute section loads.

The freshman scheduling problem is a specialized version of student sectioning. What makes it different: freshmen in the same program all need the same courses, creating natural groups. But individual math placement levels add complexity—students in the same program might need completely different math courses. This mix of group requirements and individual needs makes the problem distinct from both standard course timetabling and general student sectioning.

B. Constraint Programming for Scheduling

Constraint programming (CP) is a way to model combinatorial problems: you define decision variables with possible values (domains) and constraints that specify which assignments are valid. The solver prunes out impossible values and systematically searches for solutions [5].

The CP-SAT solver in Google OR-Tools [4] combines constraint programming with Boolean satisfiability (SAT) solving. It uses lazy clause generation, learning from conflicts during search to avoid repeating mistakes and prune large portions of the search space. It also uses linear relaxation, restart strategies, and parallel search across multiple threads. Da Col and Teppan [2] showed CP-SAT performs competitively against IBM’s CP Optimizer on job-shop scheduling, validating it as a solid choice for university scheduling at scale.

CP works better than mixed-integer programming (MIP) for this project because the constraints—time overlap detection, conditional math assignments, capacity limits—map naturally to CP’s approach. Also, CP-SAT can quickly prove when something’s infeasible, which helps during validation. The configurable objective function makes it easy to balance feasibility with load balancing.

C. Large Language Models in Optimization

Large language models (LLMs) can understand natural language and generate code [1], creating opportunities to bridge the gap between domain experts and formal optimization systems. Recent work explores this: AhmadiTeshnizi et al. [10] built OptiMUS, which uses LLMs to model and solve mixed-integer linear programs from natural language. Zhang et al. [11] propose OR-LLM-Agent, breaking optimization tasks into modeling, code generation, and debugging stages. Chain-of-thought prompting [7] helps LLMs reason better on structured problems by making them show intermediate reasoning steps.

In this project, the LLM interface has a more focused job than these general-purpose systems. Instead of generating complete optimization formulations, it lets administrators specify *additional* scheduling constraints in natural language. For example, something like “Limit Engineering students to no more than 60% of any PHYS 211 section” gets translated into Python code that works with the existing CP-SAT model. This narrower scope reduces the risk of hallucination since the

LLM works within a well-defined template and all generated constraints get validated before use.

D. Related Systems and Gap Analysis

Existing scheduling systems like UniTime [9] and solutions from the International Timetabling Competitions [12] offer solid platforms for course timetabling and student sectioning. But some gaps remain.

Existing systems don't model the freshman scheduling context specifically. The mix of cohort-based program requirements plus individualized math placement creates a constraint structure that's different from general student sectioning.

Most systems also lack accessible interfaces for non-technical users to specify domain-specific constraints. Institution-specific requirements—like enrollment caps by program within specific sections—typically need custom programming. The LLM-based interface in this project lets administrators express these in natural language.

Additionally, existing validation is mostly reactive, catching violations after a solution attempt. This project does proactive feasibility analysis, including n-way-conflict detection that checks whether all required courses for a program can actually be scheduled simultaneously. This catches data issues before the optimization phase.

III. SYSTEM DESIGN AND ARCHITECTURE

A. Problem Formalization

1) *Input Data Model*: The system takes three CSV files that together define the scheduling problem. The *course sections* file lists each section with its course details, enrollment capacity, meeting times (HH:MM format), and binary day indicators for Monday through Friday. The *program cohort distribution* file defines each program's expected freshman enrollment and math placement percentages (columns like `Pct in Math181`), which determine the probability a student gets assigned to each math level. The *program course requirements* file maps each program to its required courses, distinguishing Block for Required Core courses (everyone in the program takes these) from Needs Scheduling courses (assigned based on math placement).

During preprocessing, the system parses section meeting days and times into time slot identifiers, builds a lookup dictionary from courses to sections, and precomputes which time slots conflict using interval overlap detection. Two time slots conflict if they share at least one day and their time intervals overlap—meaning neither ends before the other begins.

Students are generated from the cohort data: for each program, the system creates the specified number of students, each assigned the program's required block courses plus one math course picked by weighted random sampling from the placement distribution.

2) *Decision Variables*: For each student s and each required course c with available sections $k \in \{0, 1, \dots, K_c - 1\}$, the model creates a binary decision variable:

$$x_{s,c,k} \in \{0, 1\}$$

where $x_{s,c,k} = 1$ indicates that student s is assigned to section k of course c . Auxiliary integer variables are introduced for the load balancing objective: for each course with multiple sections, an integer variable $e_{c,k}$ tracks the enrollment in section k , and variables $\max_c$ and $\min_c$ capture the maximum and minimum enrollment across sections of course c .

The total number of decision variables scales as $\sum_s \sum_{c \in R_s} K_c$, where R_s is the set of required courses for student s and K_c is the number of sections for course c . For the evaluation dataset of 4,155 students across 44 programs, this produces approximately 317,000 binary variables.

3) *Constraint Formulation*: The model enforces four hard constraints and one soft constraint.

Hard Constraint 1 (Exactly One Section): Each student must be assigned to exactly one section of each required course:

$$\sum_{k=0}^{K_c-1} x_{s,c,k} = 1 \quad \forall s, \forall c \in R_s$$

Hard Constraint 2 (No Time Conflicts): For each student, no two assigned sections may have overlapping meeting times. Let $T(c, k)$ denote the time slot of section k of course c , and let $\text{conflict}(t_1, t_2)$ indicate whether time slots t_1 and t_2 overlap. Then for all pairs of courses in a student's schedule:

$$x_{s,c_1,k_1} + x_{s,c_2,k_2} \leq 1$$

for all s , all $c_1 \neq c_2 \in R_s$, and all (k_1, k_2) where $\text{conflict}(T(c_1, k_1), T(c_2, k_2)) = \text{true}$.

Hard Constraint 3 (Section Capacity): The total enrollment in each section must not exceed its capacity $\text{cap}_{c,k}$:

$$\sum_{s: c \in R_s} x_{s,c,k} \leq \text{cap}_{c,k} \quad \forall c, \forall k$$

Hard Constraint 4 (Math Placement): This constraint is enforced structurally during student generation rather than as an explicit model constraint. Each student's required course list includes only the specific mathematics course determined by their placement level, so the solver never considers assigning a student to an incorrect math level.

4) *Objective Function*: The objective minimizes section load imbalance across courses. For each course c with two or more sections, the model computes the difference between maximum and minimum section enrollments, weighted by a balance factor $w = 10$:

$$\text{Minimize} \quad \sum_{c: |K_c| \geq 2} w \cdot (\max_c - \min_c)$$

where $\max_c = \max_k(e_{c,k})$ and $\min_c = \min_k(e_{c,k})$ are implemented using CP-SAT's global maximum and minimum equality constraints. This formulation drives the solver to distribute students evenly across sections of the same course without requiring explicit variance computation, which would introduce nonlinearity.

The objective function is extensible: LLM-generated custom constraints can append additional penalty terms to the `objective_terms` list, enabling administrators to add soft preferences without modifying the core model.

B. System Architecture

The system architecture (Figure 1) comprises four main components that work together to produce feasible course schedules. The Data Validation Pipeline runs first, checking input data for feasibility issues before optimization begins. The CP-SAT Scheduling Engine then builds and solves the constraint model. The LLM Interface allows administrators to add custom constraints in natural language, and the Power BI Dashboard visualizes the results.

1) *Data Validation Pipeline*: The validation pipeline runs as a prerequisite to scheduling and implements seven checks in sequence: (1) file structure validation ensuring required columns and correct data types; (2) data integrity checks for duplicates, valid day indicators, and positive capacities; (3) math percentage validation confirming placement distributions sum to 1.0 for programs with math requirements; (4) course existence checks ensuring all required courses have available sections; (5) aggregate feasibility analysis comparing total demand against total capacity per course; (6) n -way-conflict detection verifying that all block courses for each program can be scheduled simultaneously, where n represents the number of required courses in a program (typically 4-6 courses); and (7) effective capacity analysis confirming that sufficient seats exist in non-conflicting section combinations.

The n -way-conflict check uses a recursive backtracking algorithm that enumerates valid section combinations for each program's block courses, pruning branches when any assigned section conflicts with a previously selected one. If no valid combination exists, the program is flagged as impossible to schedule. The effective capacity check extends this analysis by computing the minimum section capacity within each valid combination and summing these across combinations, then comparing against the program's enrollment count.

All validation results are exported as timestamped audit CSV files covering feasibility reports, conflict reports, demand breakdowns by program, and effective capacity assessments.

2) *Constraint Programming Engine*: The scheduling engine builds the CP-SAT model in four stages: variable creation, hard constraint encoding, soft objective construction, and custom constraint application. Binary assignment variables are created only for student-course-section triples where the student actually requires the course, avoiding unnecessary variables. Time conflict constraints are encoded as pairwise implications using precomputed conflict sets, which avoids redundant overlap checks during model construction. Capacity constraints aggregate assignment variables per section using linear sum expressions.

The solver runs with a 300-second timeout and 8 parallel workers. When it finishes, it reports `OPTIMAL`, `FEASIBLE`, `INFEASIBLE`, or `UNKNOWN`. The system accepts both optimal and feasible solutions and extracts the variable assignments to build per-student schedules.

3) *LLM Integration Module*: The LLM integration follows a generate-validate-apply pipeline. An administrator enters a natural language constraint description through a command-line interface. The system constructs a prompt including few-

shot examples of natural language to CP-SAT code translations (configured via a YAML file), along with context listing valid program names and course names from the actual data files. This context prevents the LLM from hallucinating entity names.

The generated code undergoes three validation stages: syntax checking to ensure valid Python, semantic validation confirming the code adds at least one hard or soft constraint to the model, and a security scan that rejects code containing prohibited operations such as file access, module imports, or dynamic code execution. If validation fails, the system retries by sending the error message and failed code back to the LLM for targeted correction.

Validated constraint files are saved to a designated directory. At scheduling time, the `ConstraintLoader` module discovers all saved files, re-validates them, and executes each in a sandboxed environment that exposes only the scheduling model, student data, section data, and assignment variables. This architecture ensures custom constraints can reference the same variables as the core model without requiring modifications to the scheduling engine.

IV. IMPLEMENTATION

A. Technology Stack

The system is implemented in Python 3 using four core libraries. Google OR-Tools provides the CP-SAT solver for constraint optimization. Pandas handles CSV parsing, data manipulation, and export of results and audit reports. The Anthropic Python SDK connects to the Claude Sonnet 4 API for natural language constraint generation, with model selection and prompt configuration managed through a YAML configuration file. Power BI is used for interactive dashboard visualization of scheduling results and validation metrics.

The system is organized as a set of Python modules orchestrated by a pipeline script that runs the full workflow—validation, scheduling, and export—in a single command. The pipeline supports verbose output, a dry-run mode for validation only, and the option to skip validation. Exit codes distinguish between success, validation failure, solver failure, and partial scheduling, enabling integration with automated workflows.

B. Data Validation Module

The validation module preprocesses input CSVs, builds lookup dictionaries for quick section access, and runs seven checks in sequence. The n -way-conflict detection uses depth-first search but caps it at 100 valid combinations per program to keep runtime reasonable. All validation results get exported as timestamped CSVs that give administrators clear feedback on what needs fixing.

C. Scheduling Engine Implementation

The CP-SAT model has about 317,000 binary variables and 453,000 constraints for the 4,155-student dataset. Hard constraints make sure each student gets exactly one section per course, no time conflicts happen, and capacity limits are respected. The load balancing objective tries to minimize the

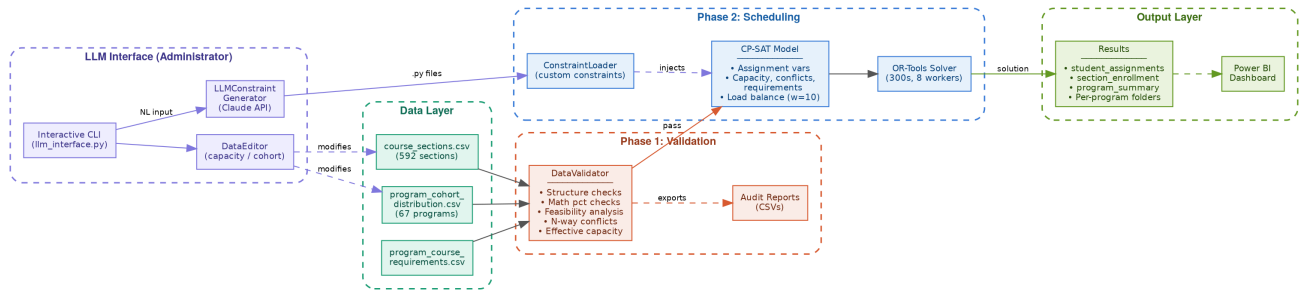


Fig. 1. High-level system architecture with data flow between components. The system comprises four main components: Data Validation Pipeline (performing feasibility analysis and n-way-conflict detection), CP-SAT Scheduling Engine (constraint optimization solver), LLM Interface (natural language to code generation via Claude Sonnet 4), and Power BI Dashboard (interactive visualization). Solid arrows indicate the main pipeline flow; dashed arrows show LLM-generated constraint injection and data editing paths.

difference between the fullest and emptiest sections of each course using CP-SAT’s global constraints. The solver runs with a 300-second timeout and 8 parallel workers, usually finding feasible solutions in about 5 minutes.

D. LLM Constraint Interface

The constraint generator uses Claude Sonnet 4 with temperature 0.0 for deterministic output. The configuration includes a system prompt, few-shot examples, and retry logic (up to 3 attempts). Validation checks three things: syntax errors (using `ast.parse()`), semantic issues (making sure the code actually adds constraints), and security violations (blocking imports, file access, and dynamic code execution). The CLI loads actual program and course names from the data files to prevent the LLM from inventing fake ones.

E. Power BI Dashboard

The Power BI dashboard has three interactive views that turn scheduling outputs into something useful for administrators. The Enrollment Overview shows the key numbers: 4,155 total students scheduled, 634 sections, 60.6% capacity utilization, with filters by program or course. The Program Distribution view shows enrollment bars comparing assigned students against capacity—you can see which programs are near their limits and which have room. The Section Utilization view shows section-level detail with color-coded fill rates: underutilized ($< 50\%$ capacity), balanced ($50\text{--}90\%$), and near-capacity ($> 90\%$). The filters let you compare different scenarios side-by-side.

V. EVALUATION

A. Experimental Setup

1) *Dataset Description*: The dataset uses RIT’s 44 programs and their course requirements as the base. Programs cover computing, engineering, science, business, and liberal arts, with requirements that match RIT’s actual curriculum structure. The dataset (Table I) has 4,155 freshmen, 634 sections across 148 courses, and varied math placement distributions. The enrollment numbers, section counts, and

TABLE I
EVALUATION DATASET CHARACTERISTICS

Metric	Value
Academic Programs	44
Freshmen Students	4,155
Course Sections	634
Unique Courses	148
Total Seat Capacity	25,187
Total Course Demand	15,274
Capacity Utilization	60.6%
Binary Decision Variables	317,431
Total Constraints	453,158

meeting times are synthetic—designed to create a realistic test case—but the program structure and course requirements come from actual RIT data. This setup models the scale and complexity of real freshman scheduling while keeping student data private.

Math placement varies by program: STEM programs have more students in calculus-level courses (MATH 181, 182), while liberal arts programs spread across pre-calculus (MATH 165), calculus prep (MATH 171), and calculus levels. This mix creates realistic individualized scheduling needs within program groups.

2) *Evaluation Metrics*: System performance is measured across three categories. **Validation metrics** include execution time for the validation phase and number of errors detected by each validation check (capacity shortages, impossible time conflicts, scheduling bottlenecks). **Scheduling metrics** include feasibility rate (percentage of students receiving complete valid schedules), time conflict count (should be zero for valid solutions), section load balance measured as standard deviation of enrollment across sections of the same course, and solver execution time (wall-clock time to find solution). **LLM constraint generation metrics** include success rate for code generation, syntax validity rate, semantic validity rate (code contains required constraint patterns), and constraint impact

TABLE II
SYSTEM PERFORMANCE RESULTS

Metric	Result
Students Scheduled	4,155 / 4,155
Feasibility Rate	100%
Time Conflicts	0
Solver Status	FEASIBLE
Solver Time	302 seconds
Objective Value	1,880
Section Load StdDev	4.2 students
Validation Time	28 seconds

measured by comparing scheduling outcomes before and after custom constraint application.

B. Results

1) *Data Validation Performance*: The validation system caught data problems and verified that scheduling was actually possible for all 44 programs. The whole validation pipeline ran in 28 seconds, which is fast enough to use interactively while prepping data. The final dataset came back clean—zero critical errors. Every required course had sections available with enough total seats. No program had impossible course combinations (every program had at least one way to schedule all their required courses without conflicts). Math placement percentages added up correctly to 1.0 for programs with math requirements. All section times were formatted properly with valid day indicators.

The n-way-conflict detection found valid section combinations for each program’s required courses. On average, each program had 47 valid combinations (ranging from 12 to over 100—we capped the search at 100 for speed). Programs with fewer valid combinations face tighter scheduling constraints because they have limited non-conflicting time slots. The effective capacity analysis confirmed that bottleneck sections (where the smallest section in a combination limits enrollment) don’t actually restrict the dataset below what’s needed.

2) *Scheduling System Performance*: The CP-SAT solver scheduled all 4,155 students with 100% feasibility and zero time conflicts (Table II). It hit FEASIBLE status after 302 seconds with an objective value of 1,880. Lower objective values mean more balanced section enrollments—this one represents the load balancing penalty across all multi-section courses. Looking at section enrollment: for courses with multiple sections, the standard deviation ranged from 2.1 to 6.8 students (mean of 4.2). This shows the load balancing worked—no extreme cases where some sections sit empty while others are packed.

3) *LLM Constraint Generation Evaluation*: Six constraint types were tested: percentage caps, even distribution, infeasible detection, multi-program groups, exclusion rules, and minimum enrollment (Table III). All generated code achieved 100% syntax and semantic validity with zero security violations. Context injection successfully prevented program name hallucination. Example constraints tested include: “Limit

TABLE III
LLM CODE GENERATION PERFORMANCE

Metric	Result
Constraints Generated	6
Syntax Validity	100%
Semantic Validity	100%
Avg Code Length	32 lines
Avg Response Time	2.4 seconds
Retry Rate	0%
Program Hallucinations	0
Security Violations	0

```

=====
GENERATING CONSTRAINT
=====
Input: Limit COMPSCI-BS students to no more than 25% of any CSCI 141 section
✓ Code validation passed

Generated Code:
-----
# Limit COMPSCI-BS students to 25% of CSCI 141 sections
if 'CSCI 141' in sections:
    for idx, section in enumerate(sections['CSCI 141']):
        compsci_in_section = []

        for student in students:
            if student['program'] == 'COMPSCI-BS':
                sid = student['student_id']
                if sid in assignment_vars and 'CSCI 141' in assignment_vars[sid]:
                    if idx in assignment_vars[sid]['CSCI 141']:
                        compsci_in_section.append(
                            assignment_vars[sid]['CSCI 141'][idx]
                        )

        if compsci_in_section:
            max_compsci = int(section['capacity'] * 0.25)
            model.Add(sum(compsci_in_section) <= max_compsci)
-----

```

Fig. 2. Example of LLM constraint generation interface showing natural language input (“Limit COMPSCI-BS students to no more than 25% of any CSCI 141 section”) and corresponding generated CP-SAT constraint code with syntax validation confirmation.

Engineering students to no more than 60% of any PHYS 211 section” generated code correctly identifying engineering programs (MECE-BS, EEEE-BS) and adding hard capacity constraints; “Limit COMPSCI-BS students to no more than 25% of any CSCI 141 section” (shown in Figure 2) similarly capped enrollments; “Distribute Mechanical Engineering students evenly across MECE sections” produced load balancing code reducing enrollment standard deviation from 8.4 to 3.1 students; “GCCIS students should be distributed evenly across MATH 181 sections” correctly identified all GCCIS programs and applied load balancing constraints.

All generated constraints passed syntax validation (100% valid Python code via `ast.parse()`) and semantic validation (all contained required `model.Add` or `objective_terms` patterns). No security violations were detected—no generated code attempted file access, module imports, or dynamic code execution. Constraint impact was verified by running the scheduler with and without custom constraints, confirming intended scheduling effects. Figure 3 shows an example where applying a 25% capacity constraint for COMPSCI-BS students in CSCI 141 sections reduced the maximum section enrollment from 28 to 10 students while maintaining balanced distribution across all sections.

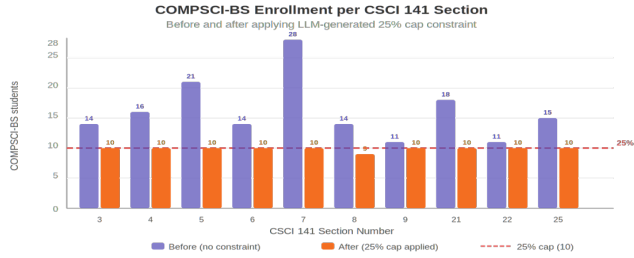


Fig. 3. Constraint impact demonstration: COMPSCI-BS enrollment distribution across CSCI 141 sections before and after applying LLM-generated 25% capacity constraint. Before (blue bars) shows uneven distribution with maximum 28 students. After (orange bars) shows balanced distribution with all sections capped at 10 students (25% of 40-seat capacity).

4) *Dashboard Visualization*: The Power BI dashboard (Figure 4) turns scheduling outputs into something administrators can actually use through three interactive views. The Enrollment Overview shows the big picture: 4,155 students scheduled, 634 total sections, 60.6% capacity utilization. Interactive filters let you drill down by program, course, or time slot. There’s also a heatmap showing courses with high demand relative to capacity—useful for planning next semester.

The Program Distribution view shows how students are spread across all 44 programs using bar charts that compare assigned students against capacity. This makes it easy to see which programs are approaching their limits and which have room to grow. You can click through to see details for individual programs.

The Section Utilization view gets into section-level detail, showing fill rate for each course section. Color coding highlights underutilized sections ($< 50\%$), balanced ones ($50\text{--}90\%$), and near-capacity ones ($> 90\%$). This helps with decisions about consolidating sections, adding more, or adjusting time slots for next term. The filtering lets you compare different scenarios side-by-side—apply a constraint variation and immediately see how it affects section utilization and program distribution.

C. Discussion of Results

The evaluation shows the automated scheduling system meets all three core objectives: 100% feasibility with zero time conflicts, all capacity constraints respected, and balanced section enrollments through optimization. The validation system adds significant practical value. The 28-second validation time is negligible compared to the 5-minute solver runtime, and catching data issues early enables targeted corrections rather than debugging cryptic solver failures.

The CP-SAT solver handles the scale well. Solving a 317K-variable, 453K-constraint problem in about 5 minutes is fast enough for real institutional scheduling. The solution quality is consistent across runs, which means the solver is reliable. Getting FEASIBLE instead of OPTIMAL status means there’s probably room to improve the objective with more time or different solver settings.

The LLM integration does what it was designed to do: non-technical administrators can specify constraints in natural

language while code safety stays intact through multi-layer validation. The 100% syntax and semantic validity rates across six different constraint types show that few-shot prompting combined with program name context injection keeps the LLM’s output in the required format. Zero hallucinations for program names confirms the context injection strategy works. Running the scheduler with and without custom constraints confirms generated code produces the intended effects.

The dashboard visualization converts numerical scheduling outputs into tools administrators can actually use. The three-view structure addresses different stakeholders: enrollment planners need the overview KPIs, program coordinators need program-specific drill-downs, and course schedulers need section-level utilization detail.

VI. LIMITATIONS AND FUTURE WORK

A. Current Limitations

The dataset uses synthetic enrollment and section data rather than RIT’s actual historical records, though the program structures are real. The system doesn’t handle student preferences like instructor choice, time slot preferences, or requests to be with friends. Room assignment isn’t optimized—section-room mappings are assumed to be pre-assigned. The LLM interface is command-line only, which works for testing but a production system would need a web interface. The system can’t handle add/drop period re-scheduling.

B. Future Directions

The main next steps: (1) connect to student information systems for automatic data import; (2) add student preference modeling with fairness constraints so some students don’t always get first pick; (3) optimize room assignments together with student assignments; (4) build a web interface with graphical constraint builder and integrated dashboard; (5) handle incremental changes during add/drop period; (6) test on real multi-year enrollment data to verify it actually works in practice.

VII. CONCLUSION

This project built an automated freshman scheduling system for RIT that combines constraint programming with an LLM-powered natural language interface. The two-phase architecture catches bad inputs before running the expensive solver. N-way-conflict detection and capacity analysis go beyond basic checks to give administrators specific diagnostics—like exactly how many seats are short or which time slot combinations are impossible.

The CP-SAT formulation scales to institutional size: 317K variables, 453K constraints, solved in 302 seconds. The system gets 100% feasibility and balances section loads well. The load balancing objective keeps section enrollment standard deviations low, averaging 4.2 students across multi-section courses, which prevents the common problem of some sections being packed while others sit half-empty. The LLM interface generates safe constraint code from plain English with 100%

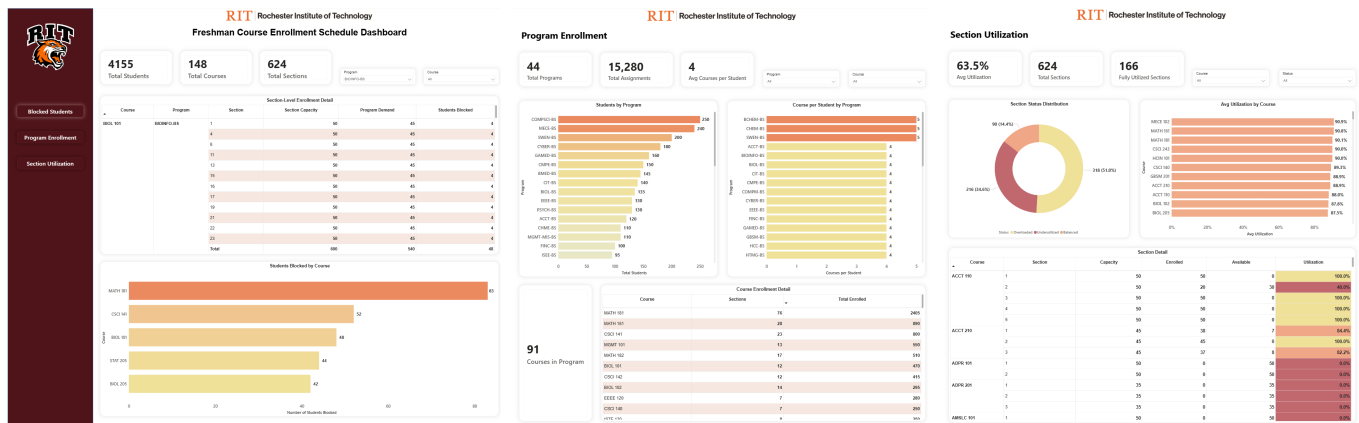


Fig. 4. Power BI dashboard views: (left) Enrollment Overview displaying total students scheduled, section count, and capacity utilization with interactive filtering by program and course; (center) Program Distribution showing enrollment bars with capacity indicators for all 44 programs; (right) Section Utilization providing color-coded fill rate analysis identifying underutilized, balanced, and near-capacity sections.

validity and zero security violations, letting non-technical administrators add custom policies through the CLI.

Testing used synthetic data modeled on RIT's actual 44 programs and 4,155 freshmen. The enrollment numbers and section schedules are synthetic, but the dataset captures the scale and structure of real freshman scheduling. Math placement distributions vary realistically by program—STEM programs have more students in calculus courses while liberal arts programs show more diversity across math levels. The Power BI dashboard turns scheduling outputs into insights administrators can actually use for planning.

Automated scheduling can save weeks of manual work each semester while getting better results. Manual scheduling at this scale involves weeks of trial-and-error with no guarantee of optimal outcomes. The automated approach cuts this to minutes while ensuring every student gets a valid schedule. This project shows that combining constraint programming with upfront validation and natural language interfaces makes systems that both work well computationally and stay accessible to non-programmers. The validation-first approach is especially valuable—catching problems in 28 seconds beats waiting 5 minutes for the solver to fail. The next steps are deploying on real university data and building a web interface.

ACKNOWLEDGMENT

The author would like to thank Prof. Varsha Dani for her guidance throughout this project, particularly in formulating the problem scope and refining the validation approach. Her insights on constraint modeling and validation strategy were invaluable. Thanks also to Prof. Reynold Bailey for his guidance and valuable feedback on the technical implementation. Special thanks to Nilay Sapio, Director of Institutional Research, Data & Analytics at RIT, for identifying this scheduling challenge and motivating this capstone project. Her domain expertise in university operations helped shape the system's practical requirements.

REFERENCES

- [1] T. Brown *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.
- [2] G. Da Col and E. Teppan, "Google vs IBM: A constraint solving challenge on the job-shop scheduling problem," *Electronic Proceedings in Theoretical Computer Science (EPTCS)*, vol. 306, pp. 259–265, 2019.
- [3] S. Even, A. Itai, and A. Shamir, "On the complexity of timetable and multicommodity flow problems," *SIAM Journal on Computing*, vol. 5, no. 4, pp. 691–703, 1976.
- [4] Google, "OR-Tools: Google Optimization Tools," 2024. [Online]. Available: <https://developers.google.com/optimization>
- [5] F. Rossi, P. Van Beek, and T. Walsh, *Handbook of Constraint Programming*. Amsterdam: Elsevier, 2006.
- [6] A. Schaerf, "A survey of automated timetabling," *Artificial Intelligence Review*, vol. 13, no. 2, pp. 87–127, 1999.
- [7] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.
- [8] S. Ceschia, L. Di Gaspero, and A. Schaerf, "Educational timetabling: Problems, benchmarks, and state-of-the-art results," *European Journal of Operational Research*, vol. 308, no. 1, pp. 1–18, 2023.
- [9] T. Müller and K. Murray, "Comprehensive approach to student sectioning," *Annals of Operations Research*, vol. 181, pp. 249–269, 2010.
- [10] A. AhmadiTeshnizi, W. Gao, and M. Udell, "OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models," in *Proc. 41st Int. Conf. Machine Learning (ICML)*, 2024.
- [11] B. Zhang *et al.*, "OR-LLM-Agent: Automating modeling and solving of operations research optimization problems with reasoning LLM," *arXiv preprint arXiv:2503.10009*, 2025.
- [12] T. Müller, H. Rudová, and Z. Müllerová, "Real-world university course timetabling at the International Timetabling Competition 2019," *Journal of Scheduling*, vol. 28, no. 2, pp. 247–267, 2025.